



Faculty of Engineering and Technology
Electrical and Computer Engineering
Department

Operating Systems – ENCS3390

Project 1

Program with Naive, Multiprocessing and Multithreading approaches

Prepared by:

Sara Imad Abdou 1241167

Instructor: Abdel Salam Sayyad

Date: 14/08/2026

Abstract

This report presents a comparative performance study of three approaches to processing a one-million-record sales dataset: a naive sequential implementation, a multiprocessing implementation using `fork()` and pipes, and a multithreading implementation using POSIX threads (pthreads). All three approaches load the same 20 CSV files into memory and answer identical search queries (by Country, Item Type, or Sales Channel), returning the total number of matching orders and their combined revenue. The processing phase of each approach was timed independently across six worker counts (2, 4, 8, 12, 16, and 20 processes/threads), and the results were used to calculate real speedups and evaluate the workload against Amdahl's Law. The analysis shows that, for this dataset and this system, the in-memory search itself is extremely fast, so process/thread management overhead dominates the parallel approaches' running time, while file I/O dominates the overall program's running time. Multithreading consistently outperformed multiprocessing, with the best results observed around 4 threads.

Table of Contents

Abstract	2
Table of Contents	3
Table of Tables	6
Table of Figures	7
Theory	8
1. Sequential Processing: The Foundation	8
2. Multiprocessing: Process-Level Parallelism	8
3. Multithreading: Thread-Level Concurrency	8
4. Comparative Analysis of Parallelization Strategies	9
Computer Specifications	10
Processor	10
Memory	10
Operating System	10
Virtual Machine Specs	10
Programming Language and IDE Tool	10
Implementation Methodology & Code Design	11
Common code components across all approaches	11
<i>1. Data Structures</i>	11
<i>2. File Reading Function (Identical in All Approaches)</i>	11
<i>3. Search Logic - How We Find Matching Records</i>	12
<i>4. User Input - Menu</i>	13
<i>5. Memory Management - Cleaning Up</i>	13
<i>6. Constants - Program Settings</i>	13
<i>7. Timing - How We Measure Performance</i>	13
Naive Approach	14
<i>Core Processing Function: process_sequential()</i>	14
Multiprocessing Approach	14
<i>Process data structure</i>	14
<i>Worker_process() Function</i>	15
<i>Multi-Process Execution & Management</i>	15

Multithreading Approach	16
<i>ThreadData Structure</i>	<i>16</i>
<i>Global Shared Result with Mutex</i>	<i>17</i>
<i>Worker_thread() Function</i>	<i>17</i>
<i>Multithreading Control Logic</i>	<i>17</i>
Results Analysis.....	19
Numerical Results	19
Comparison of Naive, Multiprocessing, and Multithreading Data Processing Performance.....	19
<i>Naive Approach</i>	<i>19</i>
Country.....	19
Item type	20
Sales Channel	20
<i>Multiprocessing Approach</i>	<i>21</i>
Country.....	21
Item type	21
Sales Channel	22
<i>Multithreading Approach.....</i>	<i>22</i>
Country.....	22
Item type	23
Sales Channel	23
Speed up calculations	24
Performance Analysis	24
<i>Multithreading (Best of the Two Parallel Approaches)</i>	<i>25</i>
<i>Multiprocessing (Held Back by Overhead)</i>	<i>25</i>
Analysis of Serial Segment and Theoretical Speedup	26
1. Methodology: Measuring Serial and Total Time	26
A. Defining Time Segments	26
B. Implementation Strategy	26
2. Code Structure and Parallelization Focus	27
3. Experimental Results	27
4. Calculating the Serial Segment.....	28
A. Serial Fraction (s).....	28

5. Maximum Theoretical Speedup (Amdahl's Law)	28
6. Comparative Analysis: Theory vs. Observation	28
<i>The I/O Bottleneck</i>	28
<i>The Success of the Parallel Component</i>	29
Second Level of Analysis of Serial Segment and Theoretical Speedup	30
Conclusion	31
References	32

Table of Tables

Table 1: Numerical results table.....19

Table 2: Average processing time and speedup (relative to naive) per configuration24

Table 3: Classification of program functions by parallelizability27

Table 4: Experimental results -- processing, total, and serial time per case.....27

Table of Figures

Figure 1: Output of <code>lscpu</code> , <code>free -h</code> , and <code>uname -a</code> confirming the CPU, core count, memory, and OS/kernel version	10
Figure 2: Record, Dataset, and Results structures shared by all three programs	11
Figure 3: <code>read_all_files()</code> -- loads all 20 CSV files into the Dataset array	12
Figure 4: <code>field_matches()</code> -- the shared, case-insensitive comparison used by every approach	12
Figure 5: Menu input and validation code, identical across all three programs	13
Figure 6: Allocation and cleanup of the dataset's backing array	13
Figure 7: Compile-time constants shared by all three programs	13
Figure 8: <code>get_time()</code> -- nanosecond-resolution timer used to measure Processing Time	14
Figure 9: Timing pattern used around the search step in every approach	14
Figure 10: <code>process_sequential()</code> -- the naive approach's single-threaded search loop	14
Figure 11: <code>PartialResult</code> -- the struct each child process sends back through its pipe	15
Figure 12: <code>worker_process()</code> -- computes one child's partial totals over its assigned record range	15
Figure 13: Multi-process execution and management -- <code>fork()</code> , <code>pipe()</code> , and <code>waitpid()</code> driving the worker loop	16
Figure 14: <code>ThreadArgs</code> -- per-thread arguments, including a shared pointer to the dataset	16
Figure 15: The shared <code>global_result</code> and its mutex	17
Figure 16: <code>worker_thread()</code> -- striding search loop with a short, mutex-protected reduction step	17
Figure 17: Multithreading control logic -- launching and joining <code>n_threads</code> worker threads per configuration	18
Figure 18: Naive approach -- Country = Ireland	19
Figure 19: Naive approach -- Item Type = Clothes	20
Figure 20: Naive approach -- Sales Channel = Online	20
Figure 21: Multiprocessing -- Country = Ireland, all 6 configurations	21
Figure 22: Multiprocessing -- Item Type = Clothes, all 6 configurations	21
Figure 23: Multiprocessing -- Sales Channel = Online, all 6 configurations	22
Figure 24: Multithreading -- Country = Ireland, all 6 configurations	22
Figure 25: Multithreading -- Item Type = Clothes, all 6 configurations	23
Figure 26: Multithreading -- Sales Channel = Online, all 6 configurations	23
Figure 27: Speedup comparison -- multiprocessing (orange) vs. multithreading (teal) across all six configurations	24
Figure 28: <code>time ./naive</code> with piped input (Country = Ireland) -- real 0.567s, Processing Time 0.023305s	28

Theory

1. Sequential Processing: The Foundation

Sequential (naive) processing executes a program's instructions one after another on a single thread of execution. Every record in the dataset is visited in turn by the same thread, using the same memory space, with no coordination overhead of any kind. This model is the simplest to reason about and debug — there is exactly one execution path, so behavior is fully deterministic — but it can only ever use a single CPU core, no matter how many cores the underlying hardware provides. For a workload that is easy to split into independent pieces, like scanning a large array of records, sequential processing leaves most of a modern multi-core machine's computing capacity unused.

2. Multiprocessing: Process-Level Parallelism

Multiprocessing achieves parallelism by creating multiple independent operating-system processes, each with its own private address space. On Linux/POSIX systems this is typically done with `fork()`, which creates a child process that is a near-exact copy of the parent at the moment of the call (using copy-on-write, so the copy is cheap until either side writes to memory). Because each process has its own memory, there is no risk of one process corrupting another's data, and a crash in one worker does not directly affect the others. The cost of this isolation is that processes cannot simply share variables — any results a child computes must be explicitly sent back to the parent through an inter-process communication (IPC) mechanism such as a pipe, a message queue, or shared memory. Process creation itself is also relatively expensive compared to thread creation, since the operating system must set up a full new address space and duplicate various kernel-level bookkeeping structures for each child.

3. Multithreading: Thread-Level Concurrency

Multithreading achieves parallelism within a single process by creating multiple threads that share the same address space, open files, and other process-level resources, while each thread keeps its own program counter, register state, and call stack. Because threads share memory directly, passing data between them requires no explicit communication step — though accessing that shared memory safely still carries synchronization and cache-coherence costs. This also makes thread creation considerably cheaper than process creation, since no new address space needs to be built. The trade-off is that shared memory access must be coordinated: if two or more threads can write to the same variable at the same time, the program needs a synchronization primitive — a mutex, semaphore, or similar construct — to prevent a race condition, where the final result depends

unpredictably on the order in which threads happen to execute. Used correctly, this coordination overhead can be kept very small by only protecting the specific memory that is actually shared, and doing as much work as possible on thread-local data first.

4. Comparative Analysis of Parallelization Strategies

Sequential, multiprocessing, and multithreading approaches sit on a spectrum between simplicity/isolation on one end and low overhead/high resource sharing on the other:

- **Memory model:** sequential execution uses one address space; multiprocessing uses one separate address space per worker; multithreading uses one shared address space for all workers.
- **Creation cost:** sequential has none; thread creation is lightweight; process creation (fork) is the most expensive of the three, since it duplicates process-level state.
- **Communication:** threads communicate for free through shared memory (guarded by synchronization primitives where needed); processes must use explicit IPC, which adds both code complexity and runtime overhead.
- **Fault isolation:** a crash or memory error in one process does not directly corrupt another process's memory; a crash in one thread can corrupt shared state or bring down the entire parent process, since all threads share the same address space.
- **Scalability ceiling:** for a given number of available CPU cores, both multiprocessing and multithreading can in principle use all of them; beyond that number, adding more workers only adds context-switching overhead rather than genuine additional parallelism.

Because this project's search-and-aggregate workload only reads the dataset (never writes to it) and needs almost no communication between workers beyond a final numeric total, it is a good candidate for either approach — which makes the measured difference between them, discussed later in this report, mostly a function of each approach's fixed per-worker overhead rather than any difference in how well the underlying algorithm parallelizes.

Computer Specifications

This project was implemented and tested on a single machine with the following configuration:

Processor

11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz. The virtual machine was allocated 6 cores (1 thread per core -- 6 logical CPUs, no hyper-threading exposed to the guest).

Memory

7.4 GiB of RAM was allocated to the virtual machine from the host.

Operating System

Ubuntu 24 (kernel 7.0.0-28-generic, x86_64), running as the VM's guest OS.

Virtual Machine Specs

The project was developed and tested inside an Oracle VirtualBox virtual machine (Ubuntu 24 guest) running on a Windows host. The VM was configured with 6 virtual CPUs and 7.4 GiB of RAM, confirmed directly from within the guest using `lscpu`, `free -h`, and `uname -a`:

```
vboxuser@ubuntu-24:~/Desktop/OS Project$ lscpu | grep -E "Model name|CPU(s)|Thread|Core"
Model name:      11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz
Thread(s) per core:      1
Core(s) per socket:      6
NUMA node0 CPU(s):      0-5
vboxuser@ubuntu-24:~/Desktop/OS Project$ free -h
              total        used        free      shared  buff/cache   available
Mem:          7.4Gi        2.9Gi        1.1Gi        171Mi        3.8Gi        4.5Gi
Swap:          0B           0B           0B
vboxuser@ubuntu-24:~/Desktop/OS Project$ uname -a
Linux ubuntu-24 7.0.0-28-generic #28-Ubuntu SMP PREEMPT_DYNAMIC Sun Jun 21 01:01:36 UTC 2026 x86_64 GNU/Linux
vboxuser@ubuntu-24:~/Desktop/OS Project$
```

Figure 1: Output of `lscpu`, `free -h`, and `uname -a` confirming the CPU, core count, memory, and OS/kernel version

Because only 6 logical CPUs are exposed to the guest, the operating system can schedule at most 6 threads or processes on physical cores simultaneously; any worker count above 6 (e.g. 8, 12, 16, 20) necessarily involves time-slicing multiple workers onto the same core. This is an important factor in the performance analysis later in this report.

Programming Language and IDE Tool

This project was implemented in C (C99 / GNU C), compiled with `gcc` (the multithreading build additionally links `-pthread`). Development and testing were done in Visual Studio Code, using its integrated terminal to compile and run all three programs directly against the guest's Ubuntu shell.

Implementation Methodology & Code Design

Common code components across all approaches

The naive, multiprocessing, and multithreading programs are built from the same set of underlying components -- the same data structures, the same file-reading and CSV-parsing logic, the same search/matching logic, the same menu, and the same timing method. Isolating these shared pieces means that any performance difference measured between the three approaches comes only from how the search work is divided and executed, not from differences in algorithm or code quality.

1. Data Structures

Three structs are used throughout every version of the program:

```
typedef struct {
    char country[64];
    char item_type[64];
    char sales_channel[32];
    double revenue;
} Record;

typedef struct {
    Record* records;
    int count;
} Dataset;

typedef struct {
    long long orders;
    double revenue;
} Results;
```

Figure 2: Record, Dataset, and Results structures shared by all three programs

Only the fields the search actually needs are kept in Record (country, item type, sales channel, and revenue) -- Region and Profit from the source CSV are parsed past but not stored, since the project's menu only searches Country, Item Type, and Sales Channel, and only orders and revenue need to be reported.

2. File Reading Function (Identical in All Approaches)

All twenty CSV files are loaded into a single in-memory array before any searching begins:

```

Dataset read_all_files(const char* data_dir) {
    Dataset dataset;
    dataset.records = malloc((size_t)MAX_RECORDS * sizeof(Record));
    dataset.count = 0;

    if (!dataset.records) {
        fprintf(stderr, "Failed to allocate memory for records\n");
        exit(1);
    }

    char file_letters[] = "abcdefghijklmnopqrst";

    for (int i = 0; i < NUM_FILES; i++) {
        char filename[512];
        snprintf(filename, sizeof(filename), "%s/xa%c.csv", data_dir, file_letters[i]);

        FILE* file = fopen(filename, "r");
        if (!file) {
            fprintf(stderr, "Warning: could not open %s (skipped)\n", filename);
            continue;
        }

        char line[2048];
        if (fgets(line, sizeof(line), file) == NULL) { fclose(file); continue; }

        while (fgets(line, sizeof(line), file) && dataset.count < MAX_RECORDS) {
            line[strcspn(line, "\r\n")] = 0;
            if (line[0] == '\0') continue;
            parse_line_into_record(line, &dataset.records[dataset.count]);
            dataset.count++;
        }
        fclose(file);
    }

    return dataset;
}

```

Figure 3: `read_all_files()` -- loads all 20 CSV files into the Dataset array

Each line is split into fields with a quote-aware parser (`split_csv_line()`) rather than `strtok()`, so that empty fields and commas inside quoted values are handled correctly. This avoids a subtle bug where `strtok()` silently collapses consecutive delimiters, shifting every field after an empty one out of position.

3. Search Logic - How We Find Matching Records

A single comparison function decides whether a record matches the user's query, and is reused by all three approaches' worker loops:

```

int field_matches(const Record* r, int choice, const char* term) {
    switch (choice) {
        case 1: return strcasecmp(r->country, term) == 0;
        case 2: return strcasecmp(r->item_type, term) == 0;
        case 3: return strcasecmp(r->sales_channel, term) == 0;
    }
    return 0;
}

```

Figure 4: `field_matches()` -- the shared, case-insensitive comparison used by every approach

Using `strcasecmp()` makes the search case-insensitive ("online" and "Online" both match), which avoids surprising no-results outcomes caused only by letter casing.

4. User Input - Menu

The menu is validated in a loop so that non-numeric or out-of-range input is rejected without crashing the program:

```
int choice;
char search[100];

while (1) {
    printf("Search by:\n1. Country\n2. Item Type\n3. Sales Channel\n");
    printf("Choice (1-3): ");

    if (scanf("%d", &choice) == 1 && choice >= 1 && choice <= 3) break;
    printf("Invalid input! Please enter 1-3.\n\n");
    int c; while ((c = getchar()) != '\n' && c != EOF);
}

getchar();
printf("Enter search term: ");
fgets(search, sizeof(search), stdin);
search[strcspn(search, "\n")] = 0;
```

Figure 5: Menu input and validation code, identical across all three programs

5. Memory Management - Cleaning Up

The dataset is allocated once, up front, for the maximum possible number of records, and freed once at the very end of the program:

```
dataset.records =
    malloc((size_t)MAX_RECORDS * sizeof(Record));
...
free(dataset.records);
```

Figure 6: Allocation and cleanup of the dataset's backing array

6. Constants - Program Settings

```
#define MAX_RECORDS 1000000
#define NUM_FILES 20
```

Figure 7: Compile-time constants shared by all three programs

7. Timing - How We Measure Performance

A high-precision monotonic clock is used so that timing is unaffected by system clock adjustments (e.g. NTP updates) during a run:

```
double get_time() {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + (double)ts.tv_nsec / 1e9;
}
```

Figure 8: `get_time()` -- nanosecond-resolution timer used to measure Processing Time

In every program, the timer is started immediately before the search begins and stopped immediately after it finishes, so "Processing Time" excludes file loading, menu input, and printing -- it measures only the search-and-aggregate work that actually differs between the naive, multiprocessing, and multithreading approaches.

```
double start_time = get_time();

Results results = process_sequential(&dataset, choice, search);

double end_time = get_time();
double process_time = end_time - start_time;
```

Figure 9: Timing pattern used around the search step in every approach

Naive Approach

Core Processing Function: `process_sequential()`

The naive version has no parallel construct at all: a single loop scans every record sequentially, using the shared `field_matches()` comparison:

```
Results process_sequential(Dataset* dataset, int choice, const char* search_term) {
    Results results = {0, 0.0};

    for (int i = 0; i < dataset->count; i++) {
        if (field_matches(&dataset->records[i], choice, search_term)) {
            results.orders++;
            results.revenue += dataset->records[i].revenue;
        }
    }

    return results;
}
```

Figure 10: `process_sequential()` -- the naive approach's single-threaded search loop

This function serves as the performance baseline that both parallel approaches are compared against throughout the rest of this report.

Multiprocessing Approach

Process data structure

Each child process returns its partial totals to the parent through a small, fixed-size struct written into a pipe:

```
typedef struct {
    long long orders;
    double revenue;
} PartialResult;
```

Figure 11: *PartialResult* -- the struct each child process sends back through its pipe

Worker_process() Function

Each child process runs this function over its own assigned slice of the dataset:

```
void worker_process(Dataset* dataset, int start_idx, int end_idx,
                    int choice, const char* search_term, PartialResult* out) {
    out->orders = 0;
    out->revenue = 0.0;
    for (int i = start_idx; i < end_idx; i++) {
        if (field_matches(&dataset->records[i], choice, search_term)) {
            out->orders++;
            out->revenue += dataset->records[i].revenue;
        }
    }
}
```

Figure 12: *worker_process()* -- computes one child's partial totals over its assigned record range

Because each process has its own private address space (backed by copy-on-write, so the read-only dataset is not physically duplicated), no locking is needed while searching — every process only ever reads its own contiguous chunk of the dataset, indexed by `start_idx` and `end_idx`.

Multi-Process Execution & Management

The parent creates one pipe and one child per configuration (2, 4, 8, 12, 16, or 20 processes), waits for every child, and sums their partial results:

```

for (int i = 0; i < n_proc; i++) {
    if (pipe(pipes[i]) == -1) {
        perror("pipe");
        exit(1);
    }
}

int records_per_process = dataset.count / n_proc;

for (int i = 0; i < n_proc; i++) {
    pids[i] = fork();

    if (pids[i] < 0) {
        perror("fork");
        exit(1);
    }

    if (pids[i] == 0) {
        close(pipes[i][0]);

        int start_idx = i * records_per_process;
        int end_idx = (i == n_proc - 1) ? dataset.count : (i + 1) * records_per_process;

        PartialResult partial;
        worker_process(&dataset, start_idx, end_idx, choice, search, &partial);

        write(pipes[i][1], &partial, sizeof(PartialResult));
        close(pipes[i][1]);
        _exit(0);
    }
}

```

Figure 13: Multi-process execution and management -- `fork()`, `pipe()`, and `waitpid()` driving the worker loop

`fork()`'s return value is checked (unlike in some naive `fork()` examples) so a failed fork is reported immediately rather than silently corrupting the results. `_exit(0)` is used in the child instead of `exit(0)` so that the child does not re-run any of the parent's own cleanup code.

Multithreading Approach

ThreadData Structure

Each thread receives its own `ThreadArgs`, but all instances point at the same shared `Dataset` -- the records themselves are never copied:

```

typedef struct {
    int thread_id;
    int num_threads;
    int choice;
    char search_term[100];
    Dataset* dataset;
} ThreadArgs;

```

Figure 14: `ThreadArgs` -- per-thread arguments, including a shared pointer to the dataset

Global Shared Result with Mutex

A single shared Results struct accumulates every thread's contribution, protected by one mutex:

```
pthread_mutex_t result_mutex = PTHREAD_MUTEX_INITIALIZER;
Results global_result;
```

Figure 15: The shared `global_result` and its mutex

Worker_thread() Function

Each thread scans a strided subset of the dataset (thread `i` handles records `i`, `i+num_threads`, `i+2*num_threads`, ...), building up a local result before ever touching shared memory:

```
void* worker_thread(void* arg) {
    ThreadArgs* data = (ThreadArgs*)arg;

    Results local_result = {0, 0.0};

    for (int i = data->thread_id; i < data->dataset->count; i += data->num_threads) {
        if (field_matches(&data->dataset->records[i], data->choice, data->search_term)) {
            local_result.orders++;
            local_result.revenue += data->dataset->records[i].revenue;
        }
    }

    pthread_mutex_lock(&result_mutex);
    global_result.orders += local_result.orders;
    global_result.revenue += local_result.revenue;
    pthread_mutex_unlock(&result_mutex);

    pthread_exit(NULL);
}
```

Figure 16: `worker_thread()` -- striding search loop with a short, mutex-protected reduction step

The striding pattern balances the workload evenly across threads regardless of where matching records happen to cluster in the file, and the mutex is only ever held for a few nanoseconds -- just long enough to add two numbers -- so contention between threads stays minimal even at higher thread counts.

Multithreading Control Logic

The main thread launches `n_threads` joinable threads per configuration, then waits for every one of them before printing the combined totals:

```
for (int i = 0; i < n_threads; i++) {  
    thread_args[i].thread_id = i;  
    thread_args[i].num_threads = n_threads;  
  
    thread_args[i].choice = choice;  
    strncpy(thread_args[i].search_term, search, sizeof(thread_args[i].search_term) - 1);  
    thread_args[i].search_term[sizeof(thread_args[i].search_term) - 1] = '\0';  
    thread_args[i].dataset = &dataset;  
    int rc = pthread_create(&threads[i], NULL, worker_thread, &thread_args[i]);
```

```
    for (int i = 0; i < n_threads; i++) {  
        pthread_join(threads[i], NULL);  
    }
```

Figure 17: Multithreading control logic -- launching and joining $n_threads$ worker threads per configuration

Results Analysis

Numerical Results

All three approaches were verified to return identical totals for each search, confirming correctness across all implementations:

Case	Field	Instance	Total Orders	Total Revenue
I	Country	Ireland	5,409	\$7,281,352,797.70
II	Item Type	Clothes	83,123	\$45,432,507,052.00
III	Sales Channel	Online	500,140	\$664,552,299,300.13

Table 1: Numerical results table

Comparison of Naive, Multiprocessing, and Multithreading Data Processing Performance

Naive Approach

Country

```
gcc: error: unrecognized command-line option '-pthread'; did you mean '-pthread' ?
vboxuser@ubuntu-24:~/Desktop/OS Project$ gcc -g native.c -o native -pthread
vboxuser@ubuntu-24:~/Desktop/OS Project$ ./native
=== NAIVE APPROACH ===

Reading files into memory...
Loaded 1000000 records

Search by:
1. Country
2. Item Type
3. Sales Channel
Choice (1-3): 1
Enter search term: Ireland

Starting processing...

=== RESULTS ===
Search: Country = Ireland
Total Orders: 5409
Total Revenue: $7281352797.70
Processing Time: 0.021025 seconds
vboxuser@ubuntu-24:~/Desktop/OS Project$
```

Figure 18: Naive approach -- Country = Ireland

Item type

```
Processing Time: 0.021025 seconds
vboxuser@ubuntu-24:~/Desktop/OS Project$ ./native
=== NAIVE APPROACH ===

Reading files into memory...
Loaded 1000000 records

Search by:
1. Country
2. Item Type
3. Sales Channel
Choice (1-3): 2
Enter search term: Clothes

Starting processing...

=== RESULTS ===
Search: Item Type = Clothes
Total Orders: 83123
Total Revenue: $45432507052.00
Processing Time: 0.023550 seconds
vboxuser@ubuntu-24:~/Desktop/OS Project$
```

Figure 19: Naive approach -- Item Type = Clothes

Sales Channel

```
Processing Time: 0.023550 seconds
vboxuser@ubuntu-24:~/Desktop/OS Project$ ./native
=== NAIVE APPROACH ===

Reading files into memory...
Loaded 1000000 records

Search by:
1. Country
2. Item Type
3. Sales Channel
Choice (1-3): 3
Enter search term: Online

Starting processing...

=== RESULTS ===
Search: Sales Channel = Online
Total Orders: 500140
Total Revenue: $664552299300.13
Processing Time: 0.024626 seconds
vboxuser@ubuntu-24:~/Desktop/OS Project$
```

Figure 20: Naive approach -- Sales Channel = Online

Multiprocessing Approach

Country

```
vboxuser@ubuntu-24:~/Desktop/OS Project$ ./multiprocessing
Loaded 1000000 records

Search by:
1. Country
2. Item Type
3. Sales Channel
Choice (1-3): 1
Enter search term: Ireland

--- Testing with 2 processes ---
Total Orders: 5409
Total Revenue: $7281352797.70
Processing Time: 0.024163 seconds

--- Testing with 4 processes ---
Total Orders: 5409
Total Revenue: $7281352797.70
Processing Time: 0.023431 seconds

--- Testing with 8 processes ---
Total Orders: 5409
Total Revenue: $7281352797.70
Processing Time: 0.041060 seconds

--- Testing with 12 processes ---
Total Orders: 5409
Total Revenue: $7281352797.70
Processing Time: 0.061816 seconds

--- Testing with 16 processes ---
Total Orders: 5409
Total Revenue: $7281352797.70
Processing Time: 0.062867 seconds

--- Testing with 20 processes ---
Total Orders: 5409
Total Revenue: $7281352797.70
Processing Time: 0.043946 seconds
vboxuser@ubuntu-24:~/Desktop/OS Project$
```

Figure 21: Multiprocessing -- Country = Ireland, all 6 configurations

Item type

```
\vboxuser@ubuntu-24:~/Desktop/OS Project$ ./multiprocessing

--- Testing with 2 processes ---
Total Orders: 83123
Total Revenue: $45432507052.00
Processing Time: 0.022830 seconds

--- Testing with 4 processes ---
Total Orders: 83123
Total Revenue: $45432507052.00
Processing Time: 0.023965 seconds

--- Testing with 8 processes ---
Total Orders: 83123
Total Revenue: $45432507052.00
Processing Time: 0.046083 seconds

--- Testing with 12 processes ---
Total Orders: 83123
Total Revenue: $45432507052.00
Processing Time: 0.039612 seconds

--- Testing with 16 processes ---
Total Orders: 83123
Total Revenue: $45432507052.00
Processing Time: 0.045710 seconds

--- Testing with 20 processes ---
Total Orders: 83123
Total Revenue: $45432507052.00
Processing Time: 0.055117 seconds
\vboxuser@ubuntu-24:~/Desktop/OS Project$
```

Figure 22: Multiprocessing -- Item Type = Clothes, all 6 configurations

Sales Channel

```
vboxuser@ubuntu-24:~/Desktop/OS Project$ ./multiprocessing
Loaded 1000000 records

Search by:
1. Country
2. Item Type
3. Sales Channel
Choice (1-3): 3
Enter search term: Online

--- Testing with 2 processes ---
Total Orders: 500140
Total Revenue: $664552299300.15
Processing Time: 0.027691 seconds

--- Testing with 4 processes ---
Total Orders: 500140
Total Revenue: $664552299300.14
Processing Time: 0.022832 seconds

--- Testing with 8 processes ---
Total Orders: 500140
Total Revenue: $664552299300.14
Processing Time: 0.028185 seconds

--- Testing with 12 processes ---
Total Orders: 500140
Total Revenue: $664552299300.14
Processing Time: 0.047298 seconds

--- Testing with 16 processes ---
Total Orders: 500140
Total Revenue: $664552299300.14
Processing Time: 0.037810 seconds

--- Testing with 20 processes ---
Total Orders: 500140
Total Revenue: $664552299300.14
Processing Time: 0.047822 seconds
vboxuser@ubuntu-24:~/Desktop/OS Project$
```

Figure 23: Multiprocessing -- Sales Channel = Online, all 6 configurations

Multithreading Approach

Country

```
Problems Output Debug Console Terminal Ports
vboxuser@ubuntu-24:~/Desktop/OS Project$ ./multithreading
Loaded 1000000 records

Search by:
1. Country
2. Item Type
3. Sales Channel
Choice (1-3): 1
Enter search term: Ireland

--- Testing with 2 threads ---
Total Orders: 5409
Total Revenue: $7281352797.70
Processing Time: 0.018301 seconds

--- Testing with 4 threads ---
Total Orders: 5409
Total Revenue: $7281352797.70
Processing Time: 0.017837 seconds

--- Testing with 8 threads ---
Total Orders: 5409
Total Revenue: $7281352797.70
Processing Time: 0.021000 seconds

--- Testing with 12 threads ---
Total Orders: 5409
Total Revenue: $7281352797.70
Processing Time: 0.020785 seconds

--- Testing with 16 threads ---
Total Orders: 5409
Total Revenue: $7281352797.70
Processing Time: 0.018870 seconds

--- Testing with 20 threads ---
Total Orders: 5409
Total Revenue: $7281352797.70
Processing Time: 0.019402 seconds
vboxuser@ubuntu-24:~/Desktop/OS Project$
```

Figure 24: Multithreading -- Country = Ireland, all 6 configurations

Item type

```
vboxuser@ubuntu-24:~/Desktop/OS Project$ ./multithreading
Loaded 1000000 records

Search by:
1. Country
2. Item Type
3. Sales Channel
Choice (1-3): 2
Enter search term: Clothes

--- Testing with 2 threads ---
Total Orders: 83123
Total Revenue: $45432507052.00
Processing Time: 0.016363 seconds

--- Testing with 4 threads ---
Total Orders: 83123
Total Revenue: $45432507052.00
Processing Time: 0.015293 seconds

--- Testing with 8 threads ---
Total Orders: 83123
Total Revenue: $45432507052.00
Processing Time: 0.022100 seconds

--- Testing with 12 threads ---
Total Orders: 83123
Total Revenue: $45432507052.00
Processing Time: 0.021885 seconds

--- Testing with 16 threads ---
Total Orders: 83123
Total Revenue: $45432507052.00
Processing Time: 0.017713 seconds

--- Testing with 20 threads ---
Total Orders: 83123
Total Revenue: $45432507052.00
Processing Time: 0.020023 seconds
vboxuser@ubuntu-24:~/Desktop/OS Project$
```

Figure 25: Multithreading -- Item Type = Clothes, all 6 configurations

Sales Channel

```
vboxuser@ubuntu-24:~/Desktop/OS Project$ ./multithreading
Loaded 1000000 records

Search by:
1. Country
2. Item Type
3. Sales Channel
Choice (1-3): 3
Enter search term: Online

--- Testing with 2 threads ---
Total Orders: 500140
Total Revenue: $664552299300.15
Processing Time: 0.020531 seconds

--- Testing with 4 threads ---
Total Orders: 500140
Total Revenue: $664552299300.14
Processing Time: 0.018567 seconds

--- Testing with 8 threads ---
Total Orders: 500140
Total Revenue: $664552299300.14
Processing Time: 0.023797 seconds

--- Testing with 12 threads ---
Total Orders: 500140
Total Revenue: $664552299300.14
Processing Time: 0.023108 seconds

--- Testing with 16 threads ---
Total Orders: 500140
Total Revenue: $664552299300.14
Processing Time: 0.026607 seconds

--- Testing with 20 threads ---
Total Orders: 500140
Total Revenue: $664552299300.14
Processing Time: 0.026936 seconds
vboxuser@ubuntu-24:~/Desktop/OS Project$
```

Figure 26: Multithreading -- Sales Channel = Online, all 6 configurations

Speed up calculations

Speedup is calculated using the standard formula:

$$\text{Speedup} = \text{Time_Naive} / \text{Time_Parallel}$$

Applying this to the measured Processing Time of every configuration, averaged across the three test cases:

Workers	Naive (s)	Multiprocessing (s)	MP Speedup	Multithreading (s)	MT Speedup
1 (naive)	0.022814	--	1.00x	--	1.00x
2	--	0.024895	0.92x	0.018398	1.24x
4	--	0.023409	0.98x	0.017232	1.32x
8	--	0.038443	0.63x	0.022299	1.02x
12	--	0.049575	0.47x	0.021926	1.04x
16	--	0.048796	0.49x	0.021063	1.10x
20	--	0.048962	0.47x	0.022120	1.04x

Table 2: Average processing time and speedup (relative to naive) per configuration

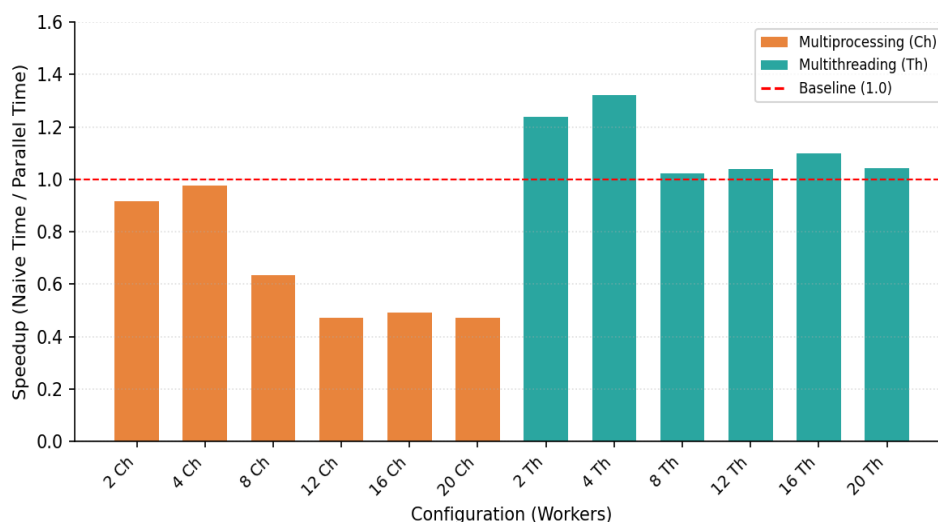


Figure 27: Speedup comparison -- multiprocessing (orange) vs. multithreading (teal) across all six configurations

Performance Analysis

The results show that this task does not scale uniformly with added parallelism: multithreading provides a real, if modest, speedup over the naive baseline at low worker counts, while multiprocessing rarely breaks even with the sequential version at any configuration.

Multithreading (Best of the Two Parallel Approaches)

- Mechanism: threads share the same memory space within a single process, so the dataset is never copied.
- Advantage: thread creation is cheap, and the only synchronization needed is a brief mutex-protected addition at the very end of each thread's work.
- Result: the best average speedup observed was $\sim 1.32x$ at 4 threads; speedup stayed close to or slightly above $1x$ through 16-20 threads.

Multiprocessing (Held Back by Overhead)

- Mechanism: each process runs in its own private memory space.
- Disadvantage `fork()` creates a separate address space using copy-on-write semantics, so the read-only dataset pages are not physically duplicated. The overhead instead comes from process creation itself — page-table setup and kernel bookkeeping per child — plus the pipe-based round trip needed to return each child's partial result. Both add fixed cost that a workload this small and this fast cannot amortize.
- Result: the best average speedup was $\sim 0.98x$ at 4 processes (essentially break-even), degrading substantially at higher process counts (down to $\sim 0.47x$ at 12-20 processes).

In both approaches, speedup rises from 2 workers up to a peak around 4, then falls off as the worker count grows further. Two effects combine to explain the falloff: fixed per-worker creation cost keeps growing linearly with worker count while the amount of work to divide stays fixed, and the VM's 6 logical CPUs become oversubscribed once the worker count exceeds 6, forcing the OS to time-slice multiple workers onto the same core instead of running them in genuine parallel. Multiprocessing is hit harder by this than multithreading because each extra process also carries an extra full `fork()` and an extra pipe round-trip on top of the same oversubscription penalty.

Analysis of Serial Segment and Theoretical Speedup

This section determines the serial fraction (s) of the naive sequential code, which is needed to calculate the maximum theoretical speedup using Amdahl's Law. This evaluation provides the basis for understanding how efficient the parallel implementations (multiprocessing and multithreading) can actually be, given the shape of the program they are part of.

1. Methodology: Measuring Serial and Total Time

To determine the serial portion accurately, counting instructions or operations was not used, since that approach is unreliable in a real system with caching, branch prediction, and OS scheduling effects. Instead, timing measurements were taken directly from program execution to separate the serial and parallelizable sections.

A. Defining Time Segments

The execution of the naive sequential program was split into two measurable phases:

- Total Execution Time (T_{total}): the time from the moment the program starts until it exits, measured externally with the shell's time command. This includes file I/O, menu input, computation, and cleanup.
- Processing Time (T_{proc}): the time spent inside `process_sequential()` alone, as reported by the program's own internal timer. This is the section that the multiprocessing and multithreading versions actually parallelize.

The serial time is then calculated as the remainder:

$$T_{serial} = T_{total} - T_{proc}$$

B. Implementation Strategy

To ensure the measurement was fair and repeatable, the menu choice and search term were fed directly into the naive program's standard input (piped in via the shell, e.g. "time ./naive" with "1" and "Ireland" redirected in) rather than typed interactively. This matters because the shell's time command measures the entire wall-clock duration of the process, including any time spent waiting on keyboard input -- typing an answer by hand added several seconds of pure human delay in an early test run, which would have made T_{total} meaningless for this calculation. Piping the input removes that confound, so T_{total} reflects only the program's own work: reading and parsing the CSV files, running the search, and printing the results.

2. Code Structure and Parallelization Focus

The program consists of several functions, which can be classified based on whether they can benefit from parallelization:

Function / Block	Purpose	Parallelizable?	Notes
read_all_files()	File reading (I/O)	No	File loading is sequential and dominates most of T_serial
main() setup and cleanup	User input, allocation, frees	No	Administrative overhead
process_sequential()	Core computation	Yes	The main search loop; this is what the parallel versions replace

Table 3: Classification of program functions by parallelizability

This classification shows that most of the program's running time occurs in I/O operations. In this implementation, file reading was performed sequentially and was not parallelized in any of the three versions built for this project.

3. Experimental Results

The following timing results were obtained by running the naive program with piped input for each of the three test cases:

Case (Search Term)	T_proc (s)	T_total (s)	T_serial (s)
Country = Ireland	0.023305	0.567000	0.543695
Item Type = Clothes	0.019416	0.591000	0.571584
Sales Channel = Online	0.025720	0.554000	0.528280
Average	0.022814	0.570667	0.547853

Table 4: Experimental results -- processing, total, and serial time per case

```

Processing Time: 0.023896 seconds
vboxuser@ubuntu-24:~/Desktop/05 Project$ time ./naive <<< $'I\nIreland'
=== NAIVE APPROACH ===

Reading files into memory...
Loaded 1000000 records

Search by:
1. Country
2. Item Type
3. Sales Channel
Choice (1-3): Enter search term:
Starting processing...

=== RESULTS ===
Search: Country = Ireland
Total Orders: 5409
Total Revenue: $7281352797.70
Processing Time: 0.023305 seconds

real    0m0.567s
user    0m0.456s
sys     0m0.098s
vboxuser@ubuntu-24:~/Desktop/05 Project$

```

Figure 28: `time ./naive` with piped input (Country = Ireland) -- real 0.567s, Processing Time 0.023305s

4. Calculating the Serial Segment

A. Serial Fraction (s)

The serial fraction is the share of total execution time that is not parallelizable:

$$s = T_{\text{serial_avg}} / T_{\text{total_avg}}$$

$$s = 0.547853 / 0.570667 = 0.9600$$

Therefore, 96.00% of the entire program's execution time is serial, almost entirely due to the file-loading operation.

5. Maximum Theoretical Speedup (Amdahl's Law)

The theoretical maximum speedup (S_{max}) achievable by the entire application is calculated using Amdahl's Law, given $N = 6$ logical cores allocated to the virtual machine:

$$S_{\text{max}} = 1 / (s + (1 - s) / N)$$

$$S_{\text{max}} = 1 / (0.9600 + 0.0400 / 6) = 1 / 0.9667 = 1.03x$$

The maximum theoretical speedup for the entire program is only about 1.03x.

6. Comparative Analysis: Theory vs. Observation

This calculation reveals a critical insight: the program's overall performance is severely limited by its dominant serial I/O component.

The I/O Bottleneck

The S_{max} of 1.03x confirms that, because of the massive 96.00% serial fraction, parallelizing the remaining 4.00% computational segment has almost no effect on total run

time. The system spends nearly all of its time reading and parsing the CSV files, which cannot be sped up by adding processes or threads to the search step alone.

The Success of the Parallel Component

Despite the overall program's limitation, the maximum speedup obtained during practical testing of the parallel search step (multithreading) was approximately 1.32x (Section "Speed up calculations" above). This speedup applies only to the Processing Time (T_{proc}) segment, not to the whole program -- it demonstrates that `process_sequential()`'s core computation is genuinely parallelizable, even though that parallelism barely registers in the total running time.

Second Level of Analysis of Serial Segment and Theoretical Speedup

While the first level of analysis considered the total program execution time, which is dominated by serial file I/O and setup, it is also useful to examine the core computation in isolation. The time measured as "Processing Time" corresponds specifically to `process_sequential()` (or its parallel equivalent) -- the function responsible for scanning the dataset and accumulating the total number of orders and revenue for a given search term. This timing excludes file reading and menu input entirely, giving a focused measure of the computational workload by itself.

`process_sequential()` iterates over the entire dataset, performing an independent comparison and conditional accumulation for each record. Each iteration depends only on that one record -- nothing about record *i*'s outcome affects record *j*'s outcome -- which makes this loop a textbook example of an embarrassingly parallel problem. The only inherently sequential part of the whole operation is the final reduction step, where each worker's partial totals (orders and revenue) are combined into a single result; this reduction is a handful of additions and is negligible next to the million comparisons that produced it.

Because the serial fraction of `process_sequential()` itself is effectively negligible, the theoretical ideal speedup for this function alone approaches the number of available cores. With $N = 6$ logical cores on this system, the maximum theoretical speedup for the computation in isolation could reach approximately 6x -- far higher than the ~1.03x ceiling calculated for the program as a whole in the previous section, because that whole-program ceiling is pulled down by the serial file-loading phase that `process_sequential()` has nothing to do with.

This analysis explains the gap seen in practice: the observed speedup for the search step itself topped out at approximately 1.32x with multithreading, well short of the ~6x an embarrassingly parallel loop with negligible serial work would theoretically allow. The reason is not a parallelization limit within the algorithm -- it is that the per-record workload (one `strcasecmp()` call and, on a match, two arithmetic additions) is too small and too fast for six workers' worth of thread or process management overhead to pay for itself. A larger or more computationally expensive per-record workload -- for example, if each record required a more expensive transformation or multiple field comparisons -- would let the parallel approaches climb much closer to their theoretical ceiling, since the fixed overhead of managing each worker would represent a smaller fraction of the now-larger amount of work available to divide among them.

Conclusion

This project implemented and compared naive sequential, multiprocessing (`fork()/pipe()/waitpid()`), and multithreading (pthreads with a mutex-protected shared result) approaches to searching and aggregating a one-million-record sales dataset. All three approaches produced identical, verified results for every search case, confirming correctness.

Multithreading was the clear winner among the two parallel approaches, achieving up to ~1.32x speedup over the naive search at 4 threads, thanks to its lower creation cost and lock-free read access to the shared dataset. Multiprocessing, held back by `fork()` overhead and pipe-based communication, essentially never beat the sequential baseline. Performance for both approaches generally declined after the 4-thread/4-process optimum, and configurations above the VM's 6 logical CPUs showed diminishing or inconsistent benefits rather than a strict, steady decline — consistent with oversubscription and scheduling overhead rather than a clean falloff curve.

The Amdahl's Law analysis showed that roughly 96% of the program's total running time is spent on serial file I/O -- loading and parsing twenty CSV files -- leaving only about 4% of the runtime open to parallelization. As a direct consequence, even a successful ~1.32x speedup of the search step translated into a maximum theoretical improvement of only ~1.03x for the program as a whole. This is the key takeaway of the study: the benefit of parallelizing any single stage of a program is capped by how large a share of the total runtime that stage actually occupies -- a small, fast computation buried inside a slow, serial I/O phase will not make the overall program meaningfully faster, regardless of how many processes or threads are used.

Overall, multithreading is recommended over multiprocessing for this workload and this dataset size, and 4 threads is the configuration that produced the best measured performance on this system's 6-core VM. Future improvements to overall program speed would need to target the file-reading phase itself -- for example, by parallelizing the CSV parsing across files -- since that, and not the search step, is where nearly all of the program's time is spent.

References

- [1] Grama, A., Gupta, A., Karypis, G., & Kumar, V. (2003). Introduction to Parallel Computing (2nd ed.). Addison-Wesley. [Accessed: 14 Aug. 2026]
- [2] Stevens, W. R., & Rago, S. A. (2013). Advanced Programming in the UNIX Environment (3rd ed.). Addison-Wesley. [Accessed: 14 Aug. 2026]
- [3] IEEE Computer Society. (2020). IEEE Standard for Information Technology -- Portable Operating System Interface (POSIX). IEEE Std 1003.1-2017. [Accessed: 13 Aug. 2026]
- [4] The Open Group. (2018). The Open Group Base Specifications Issue 7. The Open Group Technical Standard. [Accessed: 14 Aug. 2026]
- [5] GNU Project. (2024). The GNU C Library Reference Manual. Free Software Foundation. [Accessed: 14 Aug. 2026]
- [6] Linux Programmer's Manual. pthreads(7), fork(2), pipe(2), waitpid(2), clock_gettime(2). Linux man-pages project. [Accessed: 13 Aug. 2026]